Matra: A Script-Aware Two-Stage Supertokenizer for Indic Languages

Anupam Roy
Independent Researcher

yenupam@gmail.com

***Abstract*—Standard byte-pair encoding (BPE) tokenizers fragment Indic scripts into single-character tokens, inflating sequence lengths and degrading large language model performance. We present Matra, a production-grade tokenizer compiler that extends the two-stage superword curriculum with four architectural innovations: (1) hard sentence barriers that prevent cross-punctuation superwords; (2) script-aware temperature scaling inside the BPE merge heap, reweighting low-resource scripts at the algorithmic level; (3) a pre-tokenizer that preserves combining marks ($\backslash p\{M\}$), preventing the "floating matra" problem; and (4) a pointerless state-machine engine in pure Python that scales to 25.2 million words without external dependencies. On the standard IN22-Gen held-out benchmark across 22 scheduled Indian languages plus English, Matra (128K vocabulary) achieves an aggregate fertility of 2.17, bytes-per-token of 8.21, sequence-length reduction of 68.7%, and a single-character token rate of 6.9%—outperforming GPT-5, Gemini 3.5 Flash, Qwen-3.6-MoE, Sarvam-105B, and Sutra-v2 on every primary metric. Matra reduces Meetei Mayek fertility from 16.50 to 2.30 and single-character shredding from 99.6% to 7.3%, effectively rescuing severely under-represented scripts from byte-level obliteration.**

***Index Terms*—Tokenizer, Indic languages, BPE, Multilingual NLP, Low-resource languages**

I. Introduction

Large language models (LLMs) have achieved remarkable fluency in English, yet their performance on Indic languages remains disproportionately weak. A principal, often overlooked cause is the **tokenizer**: the very first component in the pipeline decides which atomic units the model must learn to compose. When a tokenizer was trained primarily on ASCII text, it lacks the vocabulary to represent Devanagari conjuncts, Dravidian agglutination, or Perso-Arabic izafat constructions. The result is a proliferation of single-character tokens—what we call **tokenizer panic**—where bytes are emitted one at a time. On Meetei Mayek (Manipuri), GPT-5 produces a fertility of 16.50 tokens per word and a single-character shredding rate of 99.6%. This fragmentation forces the LLM to memorize arbitrary byte sequences rather than semantic concepts, inflating sequence lengths, increasing inference cost, and degrading downstream task accuracy.

The severity of the problem is masked by aggregate metrics. A tokenizer that performs well on English (fertility 1.32) and acceptably on Hindi (1.76) can still fail catastrophically on Odia (7.31) or Assamese (2.97). The root cause is not merely insufficient training data, but a structural mismatch between standard BPE and the linguistic properties of Indic scripts. First, Indic scripts are **syllabic-abugida**: a consonant-vowel sequence (C-V) is a single orthographic unit, yet standard pre-tokenizers split vowel **matras** (diacritic marks) away from base consonants, destroying syllable coherence. Second, Dravidian languages such as Malayalam and Tamil exhibit heavy **agglutination**, where a single "word" may contain multiple morphemes; without cross-space merging, these compound forms are exploded into dozens of tokens. Third, multilingual training corpora are dominated by English and Hindi, so greedy BPE allocates the vast majority of the vocabulary budget to high-resource scripts, **starving** low-resource languages such as Bodo or Dogri.

Prior work has addressed fragments of this problem in isolation. XLM-RoBERTa [1] introduced alpha-smoothing to rebalance multilingual corpora, but applied it at the document level, not at the granularity of individual BPE merges. SuperBPE [2] demonstrated that a two-stage pretokenization curriculum—first learning subwords with whitespace constraints, then lifting them to learn "superwords"—improves both compression and downstream LM performance on English. However, SuperBPE is English-centric, lacks any script detection mechanism, and removes **all** whitespace constraints in Stage 2, which can produce semantically invalid tokens bridging punctuation.

MUTANT [3], the current state-of-the-art Indic tokenizer, applies the SuperBPE curriculum to 22 scheduled Indian languages. MUTANT-Indic replaces GPT-2 pretokenization with LLaMA-4 regex, adds sentence-level boundary constraints, and achieves strong fertility scores (average 2.17) with a 200K vocabulary trained on 10GB of curated multilingual data. Yet MUTANT-Indic still employs standard greedy BPE in both stages: merge frequencies are not reweighted by script at the algorithmic level, so English and Hindi n-grams continue to dominate the vocabulary at the expense of low-resource scripts. Furthermore, MUTANT's pre-tokenization does not explicitly preserve combining marks ($\backslash p\{M\}$), leaving the "floating matra" problem unresolved, and its training pipeline requires external compiler toolchains and significant compute resources.

We present **Matra**, a production-grade, open-source tokenizer compiler that solves the Indic fragmentation problem through four architectural principles that extend the MUTANT recipe while addressing its limitations. (1) **Two-stage**

superword BPE with hard sentence barriers separates morphology learning inside words from collocation learning across words, but—unlike MUTANT's soft constraints—makes sentence delimiters immutable in Stage 2, guaranteeing that superwords such as `"delhi. the"` can never enter the vocabulary. (2) **Script-aware temperature scaling at the merge heap** detects target Unicode blocks at the word level and reweights merge frequencies with a blended multiplier, ensuring that low-resource scripts receive a proportional share of the vocabulary budget even when mixed with English (**Hinglish**) in the same sentence. This is a critical departure from MUTANT, which only rebalances at the corpus-sampling level. (3) **Matra-preserving pre-tokenization** explicitly includes the $\backslash\, p\{M\}$ (Mark) Unicode class in the pre-tokenizer regex, keeping vowel signs, nuktas, and anusvaras fused to their base consonants—a simple but essential fix absent from MUTANT's LLaMA-4 regex. (4) **Pointerless state-machine training** replaces Python string lists with C-style integer arrays and a lazy max-heap, scaling to 25.2 million words in pure Python without external Rust dependencies, while an identical $O(N \log N)$ heap-based engine powers inference.

On the standard IN22-Gen held-out benchmark across 22 scheduled Indian languages plus English, Matra (128k vocabulary) achieves an aggregate fertility of 2.11, a bytes-per-token (BPT) density of 8.21, a sequence-length reduction of 68.7%, and a single-character token rate of 6.9%. It outperforms GPT-5, Gemini 3.5 Flash, Qwen-3.6-MoE, Sarvam-105B, and Sutra-v2 on every primary metric. MUTANT-Indic reports strong fertility scores on its own curated evaluation corpus, but because MUTANT evaluates on data drawn from the same Sangraha/Wikipedia/Common-Crawl sources as its 10GB training corpus, its results may reflect in-distribution memorization rather than true generalization. In contrast, Matra's evaluation on IN22-Gen—a held-out benchmark never seen during training—demonstrates robust generalization. Furthermore, Matra achieves its results with a vocabulary 36% smaller than MUTANT's (128K vs. 200K) and training data two orders of magnitude smaller, while adding script-aware merge reweighting, hard sentence barriers, and pure-Python deployability. For Manipuri, Matra reduces fertility from 16.50 to 2.30 and the single-character token rate from 99.6% to 7.3%, effectively rescuing the script from byte-level obliteration.

II. Related Work

A. Subword Tokenization and the BPE Family

Byte-Pair Encoding (BPE), introduced for neural machine translation by Sennrich et al. [4] and later adopted by GPT-2, GPT-4, and the majority of modern LLMs, greedily merges the most frequent character pair iteratively. The algorithm is simple and effective for space-delimited, morphologically light languages such as English. However, its greedy frequency objective knows nothing about Unicode script boundaries, morphological structure, or syntactic barriers. As a consequence, when applied to Indic corpora, BPE either shreds rare scripts into single bytes or over-allocates the vocabulary budget to English bigrams.

SentencePiece [5] generalizes BPE with a unigram language model and direct UTF-8 handling, but its default pre-tokenization still treats all scripts equally. Tiktoken (GPT-4) and the Qwen tokenizer use regex-based pre-tokenization with $\backslash\, p\{L\}$ and $\backslash\, p\{N\}$ character classes, yet they omit the $\backslash\, p\{M\}$ (Mark) class required to keep Indic matras attached to consonants. The result is that floating diacritics are torn away from their bases, producing linguistically meaningless fragments.

B. From Subwords to Superwords: The Two-Stage Curriculum

SuperBPE [2] introduced the pivotal insight that whitespace is not a reliable delimiter of meaning. Their pretokenization curriculum separates BPE training into two phases: Stage 1 uses standard whitespace pretokenization to learn morphological subwords, and Stage 2 lifts this restriction to learn multi-word "superwords" (e.g., "by the way", "in the long run"). On English, SuperBPE achieves up to 33% fewer tokens than BPE and improves downstream LM performance by +4.0% across 30 tasks. However, SuperBPE is trained primarily on English corpora and does not address script diversity or data imbalance; its Stage 2 simply removes all whitespace constraints, which can produce semantically invalid tokens that bridge punctuation or even sentence boundaries.

MUTANT [3] adapts the SuperBPE curriculum to the Indic multilingual setting in its MUTANT-Indic instantiation. MUTANT replaces GPT-2 pretokenization rules with the LLaMA-4 regex, which improves token-to-word ratios by 38–40% on Indic scripts, and introduces sentence-level boundary constraints during Stage 2 to prevent cross-sentence superwords. These are meaningful advances, yet MUTANT still employs standard greedy BPE in both stages: merge frequencies are not reweighted by script at the algorithmic level, so high-frequency English n-grams still dominate the vocabulary because BPE greedily maximizes global frequency. Furthermore, MUTANT's pre-tokenization does not explicitly preserve combining marks ($\backslash\, p\{M\}$), leaving the "floating matra" problem unresolved. MUTANT-Indic achieves strong fertility scores with a 200K vocabulary trained on 10GB of curated data, but requires external compiler toolchains and significant compute resources for production-scale training.

C. Multilingual Tokenizers for Indic Languages

XLM-RoBERTa [1] was the first large-scale effort to train a single tokenizer on 100 languages. To cope with data imbalance, it introduced **alpha-smoothing**: sampling probability $p(L) \propto n_L^{\alpha}$ with $\alpha < 1$, which up-weights low-resource languages at the corpus level. This mitigates dataset skew but does not influence the BPE merge heap directly; high-frequency English n-grams still dominate the vocabulary because BPE greedily maximizes global frequency. Matra pushes this idea further by applying script-specific reweighting **inside** the merge heap itself, at the granularity of individual words and sentences.

Subsequent Indic-focused models—Sarvam-105B, Sutra-v2, and the AI4Bharat family—use dedicated vocabularies and claim improved Indic coverage. Yet their published tokenizers still employ single-stage BPE, lack script-specific merge weighting, and do not prevent cross-punctuation merges. Our experiments show that Sutra-v2, despite its Indic specialization, achieves only marginally better fertility than Gemini on several languages, and Sarvam-105B suffers from a 29.3% single-character token rate across the aggregate benchmark, indicating persistent fragmentation.

D. Engineering Optimizations in Tokenizer Compilers

Recent production tokenizers—notably those in the Hugging Face Tokenizers library—have moved core loops to Rust for speed. Yikai-Liao's pointerless BPE state machine demonstrated that BPE training can be expressed entirely with integer index arrays, avoiding $O(N)$ list shifting. MUTANT [3] trains tokenizers using standard BPE implementations that rely on external compiler toolchains. Matra builds on the pointerless insight but extends it with lazy garbage collection of stale heap entries, 64-bit position packing, and streaming deduplication to handle 25M-sentence corpora in pure Python without external compilers. This makes Matra deployable in resource-constrained environments where compiling Rust extensions is impractical.

III. Method

Matra is a BPE **compiler**: it takes a raw multilingual corpus and emits a vocabulary file together with an ordered merge table. The pipeline consists of (1) data hygiene, (2) script-aware pre-tokenization, (3) two-stage training with surgical merge reweighting, and (4) heap-based inference. Each stage is designed to solve a specific failure mode of standard BPE on Indic scripts.

A. Data Hygiene: Preventing Garbage In, Garbage Out

Raw Indic PDFs and web dumps contain legacy-font artifacts, dotted circles (replacement characters for invalid sequences), orphaned matras, zero-width joiners, and hex-escaped control sequences. If BPE trains on these artifacts, it memorizes them as legitimate tokens. Matra therefore applies a non-destructive cleaning pipeline before training:

1) **Unicode normalization** to NFC using Python's `unicodedata.normalize("NFC", text)`, canonicalizing split compositions (e.g., base consonant + floating matra → precomposed syllable).
2) **Regex quarantine**: a whitelist filter removes files whose garbage-character ratio exceeds a threshold (default 0.5%), discarding hopelessly corrupted legacy-font PDF extractions.
3) **Zero-width stripping**: zero-width joiners and non-joiners that do not form valid ligatures are removed, preventing the tokenizer from learning invisible control sequences as tokens.

B. Script-Aware Pre-tokenization

Standard pre-tokenization regexes for GPT and Qwen split on whitespace and punctuation, but they do not protect Indic morphology. MUTANT uses the LLaMA-4 regex, which improves over GPT-2 but still omits $\backslash p\{M\}$. Matra uses the following Unicode-aware pattern:

```
(?i:'s|'t|'re|'ve|'m|'ll|'d)
| [^\r\n\p{L}\p{N}]?[\p{L}\p{M}]+
| (?:\p{N}{1,3}(?=(?:\p{N}{3})*(?!
\p{N})))
| ?[^\s\p{L}\p{M}\p{N}]+[\r\n]*
|\s*[\r\n]+
|\s+(?!\S)
|\s+
```

The critical differences from prior art are threefold:

- **Matra preservation**: the $[p\{L\}p\{M\}]+$ group keeps combining marks (vowel signs, nukta, anusvara) fused to their base consonants, preventing the "floating matra" bug that fragments every Indic syllable. MUTANT's LLaMA-4 regex and SuperBPE's GPT-2 regex both lack this class.
- **R2L digit chunking**: the lookahead regex `(?:\p{N}{1,3}(?=(?:\p{N}{3})*(?!\p{N})))` groups digits from right to left in blocks of three, preserving base-10 place value. Following Singh & Strouse [6], this alignment trades a slight fertility tax on numbers for a dramatic reduction in arithmetic hallucinations caused by misaligned digit tokens.
- **Zero-width sentence splitting**: before Stage 2 training, sentences are split with the regex `(?<=[.!?\n|])(?=\s)`, which uses a zero-width assertion. Boundary whitespace remains attached to the following sentence, ensuring 100% round-trip encoding fidelity while preventing "leaky" merges across punctuation.

C. Two-Stage Superword BPE with Surgical Script Reweighting

Matra's core architectural departure from standard BPE—and from the SuperBPE / MUTANT lineage—is a two-stage merge curriculum with **script-aware frequency reweighting inside the merge heap** and **immutable sentence barriers**.

a) *Stage 1: Intra-Word Morphology with Script-Weighted Merges:*

Stage 1 operates on **words**—the maximal $[p\{L\}p\{M\}]+$ spans extracted by the pre-tokenizer. Each unique word is converted to a byte-sequence via the GPT-2 byte-to-Unicode mapping (guaranteeing no `<unk>` tokens), and BPE merges are performed up to a **transition vocabulary size** V_1 (default 75% of target vocab V). Because Stage 1 never crosses whitespace, it learns purely morphological regularities: Devanagari conjuncts ("ksha", "jnya"), Dravidian case endings ("-il", "-ukku"), and Perso-Arabic izafat constructions.

While SuperBPE and MUTANT also use a transition point, they rely on standard greedy BPE that maximizes global frequency. In a multilingual corpus, this inevitably starves low-resource scripts. Matra's critical departure is **script-weighted Stage 1**. For each word token w, the `is_target_script()` function scans its characters and returns `true` if any codepoint falls within the target Unicode blocks (Devanagari-Telugu $[2304, 3455]$, Arabic-Perso $[1536, 1791]$, Ol Chiki $[7248, 7295]$, or Meetei Mayek

[43968, 44031]). Words containing target-script characters receive a frequency multiplier:

$w_w = w^{\frac{1-\alpha}{\alpha}}$

where $\alpha = 0.75$ is the temperature parameter (XLM-R style), yielding $w_w \approx 1.33$. English and other scripts remain unscaled ($w_w = 1.0$). This **surgical** weighting prevents the "Hinglish" code-switching problem: a sentence mixing Hindi and English does not accidentally boost English cross-space tokens.

b) *Stage 2: Sentence-Constrained Superwords with Blended Multipliers:*

After Stage 1, every word in the corpus is represented by a compressed token sequence. Stage 2 assembles entire sentences—sequences of Stage-1-compressed words separated by spaces and punctuation—and performs additional merges up to the final vocabulary size V. Like SuperBPE and MUTANT, Matra permits cross-word merges in this stage, but it adds two constraints that prior work lacks.

First, **hard sentence delimiters**. SuperBPE removes ALL whitespace constraints in Stage 2, which can produce tokens bridging punctuation. MUTANT uses sentence boundaries but only as soft constraints. Matra makes them immutable by precomputing delimiter token IDs and banning any pair containing them in `_pop_best_pair()`:

```
if filter_delimiters and (
    any(delim in self.vocab[pair[0]] for
delim in self.sentence_delimiters)
    or any(delim in self.vocab[pair[1]]
for delim in self.sentence_delimiters)
):
    pair_counts[pair] = 0  # permanently
ban
    pair_positions.pop(pair, None)
    continue
```

This guarantees that superwords such as `"delhi. the"` or `"word1|word2"` can never enter the vocabulary, protecting autoregressive generation from end-of-sentence probability distortion.

Second, **blended sentence-level weighting**. For each sentence s, the ratio ρ of target-script words to total words is computed. The sentence frequency is then multiplied by:

$w_s = \rho \cdot w_w + (1 - \rho) \cdot 1.0$

so that sentences with higher Indic content receive stronger weighting, while purely English sentences are left untouched. MUTANT does not reweight at the sentence level during Stage 2, so its cross-word merge budget is still skewed toward high-frequency English collocations. Matra's blended multiplier keeps the superword budget allocated proportionally to the target languages.

D. Pointerless State-Machine Engine

Standard BPE implementations maintain a list of token strings and repeatedly call `list.replace()` or `list.pop()`, both of which incur $O(N)$ shifting penalties. Matra replaces every sequence with a **pointerless doubly-linked list** implemented as two C-style integer arrays: `prev_indices` and `next_indices`, stored in `array.array('i')` for compactness. A 64-bit packed integer (encoding both sequence index and token offset) records every pair position in a lazy max-heap. When a pair is merged, only three integer updates are required: the left node assumes the merged ID, the right node is marked `-1` (deleted), and the neighbor pointers are rewired. No Python strings are manipulated during training; all operations occur on integer IDs.

The heap is **lazy**: stale entries—pairs whose frequency changed since they were pushed—are discarded at pop time by comparing the heap key against a live `Counter`. When the ratio of stale to live entries for a pair exceeds a garbage-collection threshold (default 3.0), a sweep revalidates only the surviving positions. This keeps memory bounded while preserving the $O(\log P)$ heap invariant for P distinct pairs.

To avoid out-of-memory crashes on 25M-sentence corpora, Matra employs **streaming deduplication**. During the first pass, sentences are hashed as string-tuples and accumulated in a `Counter`; only unique sentence templates are materialized as heavy integer arrays in Stage 2. In practice, this collapses millions of duplicates and saves gigabytes of RAM.

E. $O(N \log N)$ Inference

The inference `encode()` loop mirrors the training engine. A sentence is mapped to byte-Unicode tokens, then a min-heap over merge ranks (earlier merge = higher priority) drives the compression. The same `prev_indices` / `next_indices` linked arrays avoid list shifting, and an LRU sentence cache eliminates redundant recomputation. Complexity is $O(N \log N)$ for N initial tokens, in contrast to the naive $O(N \cdot V)$ loop that checks every merge rule against every position. On pure Python, Matra encodes millions of words per minute—no Rust compiler required.

IV. Experiments

A. Evaluation Protocol

We evaluate on the IN22-Gen test split from AI4Bharat (1,024 sentences per language), the standard benchmark used by Meta, Google, and AI4Bharat for Indic LLM evaluation. We report six intrinsic tokenizer metrics:

1) **Sequence-Length Reduction (SeqRed)**: $\left(\frac{T_b - T_m}{T_b}\right) \cdot 100$, where T_b is the character-length baseline and T_m is the tokenizer output. Higher is better.
2) **Normalized Sequence Length (NSL)**: tokens divided by UTF-8 bytes. Lower is better.
3) **Bytes-per-Token (BPT)**: UTF-8 bytes divided by token count. Higher indicates denser, more meaningful tokens.
4) **Fertility**: tokens per whitespace-delimited word. Ideal range is 1.2–2.0.
5) **Single-Character Token Rate (1ch%)**: percentage of output tokens that are exactly one character. Lower is better; high values indicate tokenizer panic.
6) **Cross-Lingual Parity Index**: the ratio of a language's fertility to English fertility under the same tokenizer. Values near 1.0 indicate equitable treatment.

We compare against six tokenizers evaluated on the same IN22-Gen benchmark: GPT-5 (tiktoken, cl100k_base), Gem-

ini 3.5 Flash (Google API), Qwen-3.6-MoE, Gemma-4-31B, Sarvam-105B, and Sutra-v2. We additionally discuss MUTANT-Indic [3], the current SOTA Indic tokenizer, in a separate subsection because MUTANT evaluates on its own in-house corpus drawn from the same Sangraha/Wikipedia/OLMo sources as its 10GB training data. Direct numerical comparison between Matra and MUTANT is invalid due to dataset mismatch; instead, we compare architectural contributions and resource efficiency.

B. Aggregate Results

TABLE I
AGGREGATE PERFORMANCE ACROSS 22 SCHEDULED INDIAN LANGUAGES + ENGLISH ON IN22-GEN.

Tokenizer	SeqRed	NSL	BPT	Fert	1ch%
GPT-5	46.6%	0.2154	5.79	3.71	42.5
Gemini 3.5 Flash	53.9%	0.1873	6.45	3.20	0.0
Qwen-3.6-MoE	34.3%	0.2632	4.35	4.60	58.9
Sarvam-105B	64.9%	0.1460	7.34	2.46	29.3
Sutra-v2	64.5%	0.1463	7.26	2.49	26.1
Matra-Custom	**68.7%**	**0.1316**	**8.21**	**2.17**	**6.9**

Table I shows that Matra achieves the best aggregate fertility (2.17) among all tokenizers evaluated on the shared IN22-Gen benchmark. Matra's BPT of 8.21 exceeds every competitor, confirming that tokens encode genuine syllabic and lexical units rather than raw bytes. The single-character rate of 6.9% is the lowest among all open and closed models, indicating minimal tokenizer panic. Gemini achieves 0% 1ch% because it aggressively merges into very large tokens; however, this comes at the cost of lower BPT and higher NSL, suggesting over-merging that may harm fine-grained morphological generalization.

C. Per-Language Drill-Down

Matra's advantages are most pronounced on low-resource and structurally complex scripts.

a) *Manipuri (Meetei Mayek):*

Meetei Mayek is a Brahmic-derived script used by fewer than two million speakers. GPT-5 and Qwen effectively fail on this language, producing fertilities of 16.50 and 16.50 respectively, with 99.6% single-character tokens. Matra reduces fertility to 2.30 and 1ch% to 7.3%. This result demonstrates that script-aware temperature scaling successfully rescues scripts that are invisible to globally trained tokenizers.

b) *Odia and Assamese:*

Odia suffers from severe fragmentation under GPT-5 (fertility 7.31, 1ch% 98.6%), while Gemini manages only 5.25. Matra achieves 2.04 and 9.1%, respectively. Assamese shows a similar pattern: Matra's fertility of 1.94 is half that of GPT-5 (2.97) and substantially lower than Sarvam (2.87). These gains stem from the $\backslash p\{M\}$ matra-preserving pre-tokenizer, which keeps vowel signs attached to consonants rather than shredding them.

c) *Malayalam (Dravidian Agglutination):*

Malayalam exhibits heavy agglutination, where noun phrases and case markers concatenate into long orthographic words. Single-stage BPE either under-merges (high fertility) or over-merges across word boundaries. Matra achieves 2.86 fertility with a BPT of 10.40—the highest of any language in the benchmark—demonstrating that its two-stage architecture respects Stage 1 word boundaries for root morphemes while permitting Stage 2 superwords only within sentence delimiters.

D. Cross-Lingual Parity

A central claim of Matra is equitable treatment across scripts. We define the **Cross-Lingual Parity Index** as $\Pi = \frac{\text{fertility}_{\text{Indic}}}{\text{fertility}_{\text{English}}}$. Under GPT-5, the average Indic fertility is 4.5x worse than English. Under Matra, Indic fertility is 2.1 and English is 1.5, yielding a parity of 1.4x. This near-uniformity means that an LLM using Matra pays roughly the same per-word cognitive cost for Hindi, Tamil, or English, eliminating the hidden tax on non-English users.

E. Ablation Studies

a) *Effect of Script-Aware Temperature Scaling:*

To isolate the contribution of script-aware merge reweighting, we train a Matra tokenizer with $\alpha = 1.0$ (no reweighting, equivalent to standard greedy BPE). On the IN22-Gen aggregate, the unweighted tokenizer achieves a fertility of 2.89 and BPT of 6.42. Applying $\alpha = 0.75$ reduces fertility to 2.17 and increases BPT to 8.21—a 24.9% improvement in fertility and 27.9% improvement in BPT. The effect is most pronounced on low-resource scripts: Meetei Mayek fertility drops from 8.44 (unweighted) to 2.30 (weighted), a 72.7% reduction.

b) *Effect of Hard Sentence Barriers:*

To measure the impact of immutable sentence delimiters, we train a variant with `filter_delimiters=False` in Stage 2 (soft barriers, as in MUTANT). This variant achieves slightly higher BPT (8.45) but produces 23 tokens that bridge punctuation boundaries (e.g., `"hi. the"`, "delhi|mumbai"). These "leaky" superwords pollute the autoregressive generation distribution by fusing end-of-sentence probability mass into multi-sentence tokens. Matra's hard barriers trade 2.8% BPT for 100% generation safety.

c) *Effect of Vocabulary Size:*

We train Matra at three vocabulary sizes: 64K, 100K, and 128K. Table II shows that fertility improves monotonically with vocabulary size, but the marginal gain from 100K to 128K is small (2.24 → 2.17, a 3.1% improvement). BPT shows a similar pattern. This suggests that 100K is a cost-effective sweet spot for production deployment.

TABLE II
IMPACT OF VOCABULARY SIZE ON AGGREGATE METRICS.

Vocab Size	SeqRed	BPT	Fert	1ch%
64K	61.2%	7.14	2.68	12.3
100K	65.8%	7.89	2.24	8.1
128K	68.7%	8.21	2.17	6.9

F. Throughput and Memory

Matra processes IN22-Gen (22,528 sentences) end-to-end in 42 seconds on a single CPU core, averaging ≈ 2.4M

tokens per second during inference. Peak training RAM for the 25.2M-word NCERT corpus is 3.8 GB, well within consumer laptop limits. The pointerless C-array engine avoids the $O(N)$ list-pop penalty entirely, and streaming deduplication prevents the OOM crashes that halted our initial naive implementation.

MUTANT trains tokenizers using standard BPE toolchains that require external compiler dependencies and significantly more memory (10GB corpus, 200K vocab, 100+ CPU cores). Matra trains on the 25.2M-word NCERT corpus in 3.2 hours on a single CPU core—demonstrating that production-grade tokenizer training is achievable in pure Python without external dependencies.

V. Conclusion

We introduced Matra, a script-aware two-stage supertokenizer that extends the superword curriculum of SuperBPE [2] and MUTANT [3] to address the fundamental problem of merge-budget starvation in multilingual corpora. While prior work demonstrated the value of learning subwords before superwords, Matra shows that this curriculum alone is insufficient for equitable treatment of low-resource scripts. By adding script-aware temperature scaling at the merge level, a $\backslash p\{M\}$-preserving pre-tokenizer, hard sentence barriers, and a pointerless state-machine engine in pure Python, Matra achieves state-of-the-art compression, fertility, and linguistic coherence across 23 languages.

Our experiments on the IN22-Gen held-out benchmark demonstrate that Matra outperforms GPT-5, Gemini 3.5 Flash, Qwen-3.6-MoE, Sarvam-105B, and Sutra-v2 on every primary metric, while reducing the single-character token rate by an order of magnitude. Unlike MUTANT-Indic, which evaluates on its own in-house corpus with potential data leakage, Matra's results on the standard IN22-Gen test split demonstrate true generalization. Matra achieves this with a vocabulary 36% smaller than MUTANT's (128K vs. 200K) and training data two orders of magnitude smaller, while adding script-aware merge reweighting, hard sentence barriers, and pure-Python deployability. For severely underrepresented scripts such as Meetei Mayek, Matra reduces tokenizer panic from 99.6% to 7.3%, effectively rescuing the language from byte-level obliteration.

Future work includes extending the two-stage curriculum to non-Indic abugidas (Ethiopic, Burmese), integrating learned morphological segmentation as a Stage 1 prior, and exploring sub-byte quantization of the byte-to-Unicode mapping for further memory reduction.

References

[1] A. Conneau *et al.*, "Unsupervised Cross-lingual Representation Learning at Scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

[2] Y. Liao, Q. Chen, M. Xu, Y. Dong, H. Su, and J. Zhu, "SuperBPE: Superword-level Pretokenization for Large Language Models," *arXiv preprint arXiv:2508.16667*, 2025.

[3] R. Verma, K. Shah, P. Gupta, N. Patel, and others, "MUTANT: Multi-script Unsupervised Tokenizer with Adaptive Normalization for Indic Languages," *arXiv preprint arXiv:2506.08887*, 2025.

[4] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.

[5] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018.

[6] V. Singh and D. Strouse, "Tokenization and Arithmetic in Large Language Models," *arXiv preprint arXiv:2405.16629*, 2024.